

Lambda Expressions

Functional Programming in Java

CODEHIVE

CODEHIVE

1. Introduction

Lambda Expressions were introduced in **Java 8**. They allow you to treat functionality as a method argument, or code as data. A lambda is essentially an anonymous method—a function that doesn't have a name and doesn't belong to any class.

Key Benefit: Lambdas provide a clear and concise way to represent one-method interfaces (Functional Interfaces) using an expression.

2. Syntax Patterns

The syntax of a lambda expression consists of three parts: parameters, the arrow token `->`, and a body.

`parameter → expression` (Single parameter)

`(p1, p2) → expression` (Multiple parameters)

`(p1, p2) → { code; return x; }` (Code block)

3. ArrayList Integration

Lambdas are frequently used with the Collections Framework, specifically the `forEach` method, to perform actions on Every element.

```
ArrayList<Integer> nums = new ArrayList<>();  
nums.add(5); nums.add(9);  
  
// Lambda replaces the traditional for-each loop  
nums.forEach((n) -> System.out.println(n));
```

4. Functional Interfaces

A lambda can only be used where a **Functional Interface** is expected. This is an interface with exactly one abstract method (e.g., `Consumer`, `Runnable`, or custom interfaces).

```
import java.util.function.Consumer;

// Storing a lambda in a variable
Consumer<Integer> printMethod = (n) -> System.out.println(n);
numbers.forEach(printMethod);
```

5. Custom Lambda Parameters

You can define your own interfaces to accept lambdas as parameters in your methods, enabling behavior injection.

```
interface StringOp { String run(String s); }

public static void printIt(String s, StringOp op) {
    System.out.println(op.run(s));
}

// Usage
printIt("Hello", (s) -> s + "!");
```

6. Lambda vs. Anonymous Classes

Before Java 8, we used Anonymous Inner Classes. Lambdas are much more readable and have less syntax overhead.

Anonymous Class:

```
Greeting g = new Greeting() { public void say() { ... } };
```

Lambda Expression:

```
Greeting g = () -> System.out.println("Hi!");
```

7. Lexical Scoping

Lambdas can access "effectively final" variables from their enclosing scope. This means you can use a variable inside a lambda if its value never changes after being initialized.

```
String prefix = "User: ";  
// prefix = "Admin: "; // Error! Would break effectively final  
nums.forEach(n -> System.out.println(prefix + n));
```


8. Method References

In many cases, a lambda just calls an existing method. Java provides a shorthand for this called **Method References** using the `::` operator.

Lambda: `(n) -> System.out.println(n)`

Method Reference: `System.out::println`

9. Summary & Review

- Lambdas are short blocks of code used for functional interfaces.
- They significantly reduce "boilerplate" code.
- Structure: (parameters) -> { body }.
- Mainly used with the **Stream API** and **Collections**.

© 2026 CodeHive Academy | Modern Java Programming